

Conversational AI — AIMLCZG521

Sessions 1–7 | 30 Marks | Topics: Foundations · Embeddings

Revision Sheet · BITS-Pilani · Dr. Bharathi R

Prompting · RLHF · R

■ ■ Exam Structure — 30 Marks (5 Questions × 6 Marks Each)

Q.No.	Topics Covered	Marks	What to Expect
Q1	Sessions 1 & 2: Foundations + Embeddings	6	Tokenization (BPE), Context Windows, Transformer Architecture, Encoder Models, Pooling Strategies, Similarity Metrics
Q2	Sessions 2 & 3: ANN Methods + Hybrid Search	6	HNSW, IVF, PQ, BM25 formula, RRF calculation, Dense vs Sparse comparison
Q3	Session 4: Model Landscape + Cost Engineering	6	Dense/MoE/SLM/SSM models, Quantization formats, LoRA/QLoRA, GPU Memory, Token Pricing, Prompt Caching
Q4	Session 5: Function Calling, ReAct, Prompting	6	JSON Schema, Parallel calling, ReAct loop, CoT vs ToT vs APE, Safety Mechanisms
Q5	Sessions 6 & 7: Instruction Tuning, RLHF, PEFT	6	Self-Instruct pipeline, PPO vs DPO vs GRPO, LoRA math, QLoRA memory, PEFT taxonomy

■ EXAM: All 3 question types: Numerical Problems, Scenario-Based, Conceptual. Complex formulas will be provided.

■ SESSION 1: Foundations of Conversational AI

What is Conversational AI?

Definition: Any AI system that engages humans through natural language to understand intent, retain context, retrieve knowledge, and deliver information or take real-world action.

Core Capabilities: NLU (Natural Language Understanding) → Dialogue Management → NLG (Natural Language Generation)

Tokenization — The Foundation

Algorithm	Used By	Key Feature
BPE (Byte-Pair Encoding)	GPT family	Merges most frequent adjacent character pairs iteratively
SentencePiece	Multilingual models	Language-agnostic subword tokenization
WordPiece	BERT	Maximizes language model probability of training data

BPE — How it Works (Exam Favourite)

- Step 1: Split corpus into characters. Count word frequencies.
- Step 2: Find most frequent adjacent pair (e.g. "u"+"g" → "ug" appears 20 times).
- Step 3: Merge that pair and add to vocabulary. Repeat until desired vocabulary size.

Example: corpus = {hug:10, pug:5, pun:12, bun:4, hugs:5}

Merges: (u,g)→"ug" | (u,n)→"un" | (h,ug)→"hug"

"bug" → ["b","ug"] | "mug" → ["[UNK]","ug"] | "unhug" → ["un","hug"]

■ EXAM: "unhug" → ["un","hug"] using the 3 merge rules above. Expect BPE trace questions.

Why Tokenization Matters

Impact	Detail
Cost	API pricing per token → avg conversation 800–1,200 tokens → GPT-4o ≈ \$0.01–0.03/conv
Context Window	Limits conversation length (200K tokens ≈ 150K words)
Latency	More tokens = slower response
Quality	Domain-specific terms may tokenize poorly (e.g. "GPT-4" → 4 tokens)

Context Windows — 2025

Model	Context Window	Notes
GPT-4 Turbo	128K (~96K words)	Standard production
Claude 3.5 Sonnet	200K (~150K words)	Extended conversations
Gemini 1.5 Pro	1M (~750K words)	Entire codebases
Emerging Models	2M+ tokens	Specialized use cases

■ EXAM: "Lost in the middle" problem — models struggle with info in long contexts. Solution: RAG + Memory Systems.

LLM Capabilities vs Limitations

✓ What LLMs Do Well	✗ LLM Limitations
• Natural conversation • Intent understanding • Context retention (multi-turn) • Summarization & entity extraction • Sentiment analysis	• No real-time data (training cutoff) • Hallucinations (plausible but false info) • No actions (no access to external systems) • No persistent memory • Calculation errors • Inconsistent answers

Agentic Architecture — LLM Limitation → Solution

LLM Limitation	Agentic Solution	Example
No real-time data	Tool calling (APIs, web search)	Check current flight prices
Hallucinations	RAG (Retrieval-Augmented Generation)	Ground answers in company docs
Can't take actions	Function calling	Book appointment, send email
No memory	External memory systems	Remember user preferences
Calculation errors	Calculator tool, code execution	Precise math computations

7-Stage Agent Pipeline (Request → Response)

Stage	Function	Key Activities
1. ■ REQUEST	Receive & validate	Parse intent, extract entities, input sanitization
2. ■ ROUTING	Intent classification	Classify intent type, determine tools, route to sub-agent
3. ■ REASONING	Planning	Break into steps, identify info gaps, plan tool sequence
4. ■ TOOL INVOCATION	Execute APIs	External API calls, DB queries, code execution
5. ■ MEMORY	Context storage	Store conversation history, update preferences
6. ■ SAFETY	Guardrails	Check toxic content, verify accuracy, PII redaction
7. ■ RESPONSE	Output	NLG, format for channel (text/voice/UI), send to user

■ EXAM: Banking agent example: User asks "balance + transactions" → 7 stages mapped step by step. Know each stage's role.

■ SESSION 2: Vector Embeddings & Encoder Models

Transformer Architecture — Key Blocks

Block	Function
Embeddings + Positional Encoding	Token IDs → dense vectors + position info
Multi-Head Self-Attention	Parallel attention over different subspaces (h heads)
LayerNorm + Residual	Add & normalize after each sub-layer
Feed-Forward Network (FFN)	Two linear layers with ReLU; applied per position
Nx Layers	Stack of N identical blocks (BERT-base: 12, BERT-large: 24)
Linear + Softmax (Decoder)	Project to vocab size → probability distribution

Self-Attention Mathematics — Core Formula

Attention(Q, K, V) = softmax(QK^T / √d_k) · V

• Q (Query): What am I looking for? Q = X·W_Q

- **K (Key):** What do I contain? $K = X \cdot W_K$
- **V (Value):** What info do I carry? $V = X \cdot W_V$
- **d_k:** Dimension of keys (typically 64). Scaling prevents vanishing gradients for large d_k.

Self-Attention vs Cross-Attention

Mechanism	Location	Q Source	K & V Source	Example
Self-Attention	Encoder & Decoder	Current layer input	Current layer input	"bank" sees "river" → means riverbank
Cross-Attention	Decoder only	Decoder (current state)	Encoder (final output)	Translating "Mangoes" → "Aam"

Encoder vs Decoder vs Encoder-Decoder

Architecture	Direction	Training Objective	Best For	Popular Models (2025)
Encoder-Only	Bidirectional (sees full input)	MLM (Masked Language Modeling)	Classification, NER, Semantic Search, Embeddings	BERT, RoBERTa, DeBERTa, ALBERT
Decoder-Only	Unidirectional (causal)	Autoregressive (predict next token)	Text generation, Chatbots, Code synthesis	GPT-4, Claude 3/4, Llama 3, Mistral
Encoder-Decoder	Bidirectional encode, causal decode	Seq2Seq	Machine Translation, Summarization	T5, BART, FLAN-T5, mT5

■ **EXAM: Encoders: best for understanding & embeddings. Decoders: best for generation & ConvAI. Recent paper: fine-tuned decoders match encoder performance on classification.**

BERT Embedding Pipeline — 5 Steps

Step	What Happens	Key Detail
1. Tokenization	Wrap with [CLS] + [SEP]	[CLS] at pos 0 summarizes sentence; [SEP] marks end
2. Token Embeddings	Lookup 30,000×768 table	Static vectors — same word → same row, no context yet
3. Positional Embeddings	Add position vector	$input_emb = token_emb + pos_emb$ (elementwise)
4. 12 Transformer Layers	Self-attention through all layers	Layer 1: syntax; Layer 4: entities; Layer 12: full semantics
5. Pooling Strategy	Extract sentence vector	CLS pooling or Mean pooling or Max pooling

Pooling Strategies

Strategy	Formula	Best For
CLS Pooling	$v = h[CLS]$	Classification, sentiment, NLI (default in BERT, RoBERTa)
Mean Pooling	$v = (1/n) \sum h_i$	Semantic similarity, search, RAG (used by SBERT, BGE, GTE)
Max Pooling	$v_i = \max(h_{i1}, h_{i2}, \dots, h_{in})$	Some retrieval models

■ **EXAM: Mean pooling often outperforms CLS for sentence embeddings. Key insight: "bank" gets different embedding in "river bank" vs "bank account" — contextual embeddings, unlike Word2Vec (static).**

Embedding Training Objectives

Objective	Core Idea	Formula	Models
Contrastive Learning	Similar items close, dissimilar far in vector space	$Loss = -\log(\exp(\text{sim}(a,p)) / \sum \exp(\text{sim}(a,n)))$	SimCSE, SBERT, E5, GTE, BGE
MLM (Masked LM)	Predict masked words using surrounding context	$Loss = -\sum \log P(\text{masked_word} \text{context})$	BERT, RoBERTa, SpanBERT
RetroMAE	Reconstruct entire sentence from heavily masked version (50–70% masked)	$Loss = \text{MSE}(\text{original_emb}, \text{reconstructed_emb})$	RetroMAE, RetroMAE-v2

Embedding Models — Production Reference

Model	Provider	Dims	Context	Type	Best For
bge-large-en-v1.5	BAAI	1,024	512	Open (BERT)	General purpose, high quality
gte-large-en-v1.5	Alibaba	1,024	8,192	Open (BERT)	Long documents
e5-mistral-7b-instruct	Microsoft	4,096	32,768	Open (Mistral)	Very long context, instruction-following
text-embedding-3-large	OpenAI	3,072	8,191	API	Highest quality, \$0.13/1M tokens
embed-english-v3.0	Cohere	1,024	512	API	Compression support, \$0.10/1M tokens

✓ TIP: Start with bge-large-en-v1.5 for general use. Use gte-large or e5-mistral for docs >1000 tokens.

Vector Similarity Metrics

Metric	Formula	Range	Interpretation	Best For
Cosine Similarity	$\cos(\theta) = (A \cdot B) / (A B)$	$[-1, 1]$	1=identical, 0=orthogonal, -1=opposite. NOT sensitive to magnitude.	Text, high-dimensional data, semantic search
Euclidean (L2)	$d(A,B) = \sqrt{\sum (A_i - B_i)^2}$	$[0, \infty)$	0=identical. Sensitive to magnitude. Normalize first!	Image data, geometry
Dot Product	$A \cdot B = \sum (A_i \times B_i)$	$(-\infty, \infty)$	If vectors normalized: = cosine similarity	Fast computation when vectors are normalized

■ EXAM: If $|A|=|B|=1$ (normalized), dot product = cosine similarity. Cosine measures angle, L2 measures distance.

■ SESSION 3: ANN Search Algorithms & Hybrid Retrieval

The Computational Challenge

Problem: 10M vectors × 768 dimensions → 7.68 billion multiply-add operations per query → 7.68 seconds at 1B ops/sec = IMPRACTICAL

Solution: Approximate Nearest Neighbor (ANN) — trades tiny accuracy loss (95–99% recall) for **100–1000× speedup**

ANN Indexing Strategies — Master Comparison

Algorithm	Type	Search Time	Memory	Build Time	Recall	Key Params
Linear Scan	Brute force	$O(n \cdot d)$	$O(n \cdot d)$	$O(1)$	100%	—
HNSW	Graph-based	$O(\log n)$	$O(n \cdot d)$ HIGH	$O(n \log n)$	95–99%	M, ef_construction, ef_search
IVF	Partition/Cluster	$O((n/k) \cdot d)$	$O(n \cdot d)$	$O(n \cdot d \cdot i)$	90–95%	nlist, nprobe
PQ	Compression	$O(n \cdot m)$	$O(n \cdot m)$ LOW	$O(n \cdot d \cdot i)$	70–85%	M subvectors, k centroids
IVF+PQ	Hybrid	$O((n/k) \cdot m)$	$O(n \cdot m)$ LOW	$O(n \cdot d \cdot i)$	85–92%	nlist, nprobe, m
HNSW+PQ	Hybrid (best)	$O(\log n)$	$O(n \cdot m)$	$O(n \log n)$	90–96%	M, ef, m

■ EXAM: HNSW: best recall, high memory. PQ: least memory, lower recall. HNSW+PQ = production sweet spot. Libraries: FAISS, ScaNN, ANNOY.

HNSW — Hierarchical Navigable Small World

- **Structure:** Multi-layer graph. Layer 2 (top) = sparse, long-range connections. Layer 0 (bottom) = all vectors, dense connections.
- **Search:** Enter top layer → greedy walk toward query → when stuck, descend to next layer → beam search at Layer 0.
- **Complexity:** $O(\log n)$ — small world property means average path length is logarithmic.

HNSW Parameters

Parameter	Range	Effect	Default
M (connections/node)	16–64	Higher M = better recall + more memory + slower inserts	16

Parameter	Range	Effect	Default
ef_construction	100–200	Higher = better index quality, slower build	200
ef_search	50–500	Higher = better recall, slower query. Tune at query time.	100

HNSW Memory: ~3,136 bytes/vector (768-dim, M=16)

Vector: 768×4 = 3,072 bytes | Layer info: 1 byte | Connections: ~64 bytes | Total: ~3,136 bytes

✓ **TIP: Budget 1.5–2.0× raw vector memory for production HNSW.**

IVF — Inverted File Index

- **Partition Strategy:** k-means clustering into nlist coarse centroids. Each vector assigned to nearest centroid.
- **Search:** Query compared to all centroids → top nprobe clusters selected → only those vectors searched.
- **Complexity:** $O(K \cdot d + N \cdot d / K) \approx 100\times$ faster than brute-force at K=100, N=10M.

IVF Numerical Example — nlist=3, nprobe=1

Query q=(5.0,5.0) | Centroids: C1(1.25,1.00), C2(4.90,5.25), C3(8.85,1.15)

Dist to C1=5.48, C2=0.26✓, C3=5.44 → Search only C2 cluster

Result: v3(5.0,5.1) at dist 0.10 – Only 2 of 6 vectors scanned!

IVF Parameters

Parameter	Rule of Thumb	Effect
nlist	$4\sqrt{n}$ (e.g. 1M vectors → nlist≈1,000–4,000)	More clusters = finer partitions, need more nprobe
nprobe	10–20 typical; nprobe=K → exact search	Higher = better recall, slower. nprobe=10 gives >95% recall

Product Quantization (PQ)

- **Core Idea:** Split d-dimensional vector into M subvectors. Quantize each subvector independently using k=256 centroids (codebook).
- **Encoding:** Replace each subvector with 1-byte ID (0–255) of nearest centroid → 128× compression (512 bytes → 4 bytes for 128-dim, M=4)
- **Search (ADC):** Pre-compute distance table (query subvector to all 256 centroids). Approximate distance = table lookup sum.

PQ Numerical Example — Memory Savings

1 Billion vectors × 768 dims × 4 bytes = 2.88 TB → with PQ (M=96 subvectors, 1 byte each) → ~96 GB

5 vectors × 8 dims: Original = 160 bytes → PQ (M=2, k=2) = 10 bytes + 64 byte codebook = 74 bytes total

■ **EXAM: PQ Step sequence: 1) Train codebooks (k-means per subspace) 2) Encode vectors → compact IDs 3) ADC for fast search.**
Key: codebook cost is fixed overhead, negligible at scale.

Sparse Retrieval — BM25

Why Dense Fails → When to Use Sparse

✓ Dense (Embeddings)	✗ Sparse (BM25)
✓ Semantic meaning ✓ Synonyms/paraphrases ✓ Context understanding ✓ Long queries ✓ Q&A, RAG	✓ Exact keyword matching ✓ IDs, numbers, error codes ✓ Rare/out-of-vocabulary terms ✓ Log messages, code snippets ✓ Unseen terms (always works)

TF-IDF → BM25 Evolution

$TF\text{-}IDF(t,d) = (f_{\{t,d\}}/|d|) \times \log(N/df_t)$

- **TF-IDF Limitations:** Linear TF (no saturation), no length normalization, bag-of-words (ignores order), no term saturation.

$BM25(Q,D) = \sum IDF(q_i) \cdot [f(q_i,D) \cdot (k+1)] / [f(q_i,D) + k \cdot (1-b+b \cdot |D|/avgdl)]$

$IDF(q_i) = \log[(N - df_i + 0.5) / (df_i + 0.5) + 1]$

BM25 Parameters

Parameter	Default	Range	Effect
k	1.5	[1.2, 2.0]	TF saturation. Higher = more weight on term frequency
b	0.75	[0, 1]	Length normalization. 0=no norm, 1=full norm

BM25 Step-by-Step Numerical (Exam Critical)

Corpus: 1000 docs, avgdl=50. Doc D: 12 terms. Query: "machine learning". $k=1.5$, $b=0.75$

IDF("machine"): $\log[(1000-300+0.5)/(300+0.5)+1] = \log[3.331] = 1.203$

IDF("learning"): $\log[(1000-400+0.5)/(400+0.5)+1] = \log[2.499] = 0.916$

norm = $1 - 0.75 + 0.75 \times (12/50) = 0.43$

TF("machine", f=2) = $[2 \times 2.5] / [2 + 1.5 \times 0.43] = 5.0 / 2.645 = 1.890$

TF("learning", f=3) = $[3 \times 2.5] / [3 + 0.645] = 7.5 / 3.645 = 2.058$

BM25 Score = $1.203 \times 1.890 + 0.916 \times 2.058 = 2.274 + 1.885 = 4.159$

TF-IDF Score (same query) = 0.572 → BM25 is MUCH higher due to saturation + length norm

■ EXAM: BM25 vs TF-IDF: BM25 scores higher because: (1) short docs penalized less, (2) TF saturates, (3) probabilistic foundation.

Hybrid Search — RRF (Reciprocal Rank Fusion)

Problem: Dense scores ∈ [0,1], BM25 scores ∈ [0,∞) → cannot simply add them.

$$RRF(d) = \sum_{r \in \text{rankings}} \frac{1}{(k + \text{rank}_r(d))}$$
 where $k = 60$ (empirical from TREC)

RRF Numerical Example

Doc A: Dense rank=1, BM25 rank=2 → $RRF = 1/61 + 1/62 = 0.03252$

Doc B: Dense rank=3, BM25 rank=1 → $RRF = 1/63 + 1/61 = 0.03226$

Doc C: Dense rank=2, BM25 rank=5 → $RRF = 1/62 + 1/65 = 0.03151$

Result: Doc A wins – consensus across BOTH systems.

RRF Properties

Property	Meaning
Score-agnostic	Uses only rank order, not raw scores. Works regardless of scale.
Bounded	Score ∈ [0, num_rankings/k]. Predictable output.
Symmetric	All rankings equally weighted.
Robust	Works even if one ranking is poor.
No Training	No labeled data. k=60 is the only hyperparameter.

Hybrid Performance vs Single-System

Pipeline	Latency	Recall	Precision
BM25 only	20ms	70%	60%
Vector only	50ms	75%	65%
Hybrid (BM25+Vector+RRF)	70ms	90%	75%

■ EXAM: Why k=60? Without k: 1/1=1.0 vs 1/2=0.5 (too steep). With k=60: 1/61~1/62 (gentler drop, prevents rank-1 dominating).

■ SESSION 4: Model Landscape & Cost Engineering

2025 LLM Architecture Landscape

Architecture	Key Models	Context	MMLU	Cost/1M tok	Best For
Dense Transformer	GPT-4o, Claude 3.7, Llama 3.3 70B, Gemini 2.0	128K–1M	86–90%	\$3–\$30	Complex reasoning, multimodal
Mixture-of-Experts (MoE)	DeepSeek-V3, Mixtral 8×22B, Qwen2-57B, Jamba-1.5	32K–256K	80–88%	\$0.3–\$2	Cost-effective at scale
Small LMs (1–14B)	Phi-4, Gemma 3, Qwen2.5-7B, Llama 3.2	4K–128K	72–85%	\$0.05–\$0.3	Routing, edge, classification
State Space Models	Mamba-2, RWKV-6, Jamba-1.5 (hybrid)	1M+	68–75%	Variable	Ultra-long context tasks

Mixture of Experts (MoE) — How It Works

- **Routing:** $\text{router_logits} = W_{\text{router}} \cdot x \rightarrow \text{TopK selection} \rightarrow \text{output} = \sum (w_{\text{expert}} \times \text{Expert}_{\text{expert}}(x))$
- **DeepSeek-V3:** 671B total params, 37B active per token → 94.5% compute saving
- **Load Balancing:** Auxiliary loss $L_{\text{aux}} = \alpha \times \text{load_balance_loss}$ ($\alpha=0.001$) prevents expert collapse

Cost Reduction = 1 - (Active / Total) | DeepSeek-V3: 1 - 37B/671B = 94.5%

State Space Models (SSM) — Linear Complexity

Feature	Transformer	Mamba-2 (SSM)
Complexity	$O(n^2 \cdot d)$	$O(n \cdot d)$
Memory (128K seq)	~16 GB attention map	Constant per step $O(d_{state})$
Context scaling	Impractical >128K	1M+ tokens efficiently
Core equations	$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$	$h_t = A \cdot h_{t-1} + B \cdot x_t \mid y_t = C \cdot h_t + D \cdot x_t$

■ **EXAM: SSM limitation: weaker in-context learning (multi-hop QA). Hybrid solution: Jamba-1.5 = 50% Mamba + 50% Attention.**

Quantization Formats — Memory vs Quality

Format	Bits	Memory (7B)	Memory (70B)	Quality	Speed	Best Use
FP32	32	28 GB	280 GB	100%	1×	Training from scratch
BF16	16	14 GB	140 GB	~99.9%	1.5–2×	LLM inference standard (H100/A100)
FP8 (E4M3)	8	7 GB	70 GB	~99.5%	2–3×	H100 native — 2025 default
INT8	8	7 GB	70 GB	~99%	2–4×	Production on A10G, V100, A100
NF4 / GPTQ-4bit	4	3.5 GB	35 GB	~98%	3–6×	QLoRA fine-tuning, edge deployment
INT4 / AWQ	4	3.5 GB	35 GB	~98.5%	4–6×	Mobile/edge — best quality at 4-bit

■ **EXAM: Production rule: INT8 = 4× memory reduction, 2–4× speed, <1% quality loss. AWQ vs GPTQ: AWQ protects top 1% salient weights → better quality.**

GPU Memory Formula

$M_{total} = M_{weights} + M_{KV-cache} + M_{activation}$

$M_{weights}$: Parameters × bytes/param (e.g. 7B × 2 bytes = 14 GB BF16)

$M_{KV} = 2 \times layers \times heads \times d_{head} \times seq_len \times batch \times bytes$

$M_{activation} \approx 10\text{--}20\%$ of $M_{weights}$

Hands-on Memory Calculation — 175B Model

$M_{weights}$: 175B × 2 bytes = 350 GB

M_{KV} : $2 \times 96 \times 96 \times 128 \times 4096 \times 4 \times 2 = 96\text{ GB}$

$M_{activation}$: $4 \times 4096 \times 12,288 \times 96 \times 2 = 38\text{ GB}$

TOTAL = 350 + 96 + 38 ≈ 484 GB → 6–7× A100 80GB (or 2× H100 with FP8)

KV-Cache — Why It Matters

- **Without KV-Cache:** Recompute all Q,K,V for every new token → $O(n^2)$ total work.
- **With KV-Cache:** Cache K,V once, reuse for all future tokens → $O(n)$ total work. Q always recomputed (changes per token).
- **Warning:** At 128K context, batch=8 → KV-cache can exceed 137 GB (more than model weights!). Use GQA + PagedAttention + FlashAttention-3.

LoRA — Low-Rank Adaptation

$W_{new} = W_{frozen} + \Delta W$ where $\Delta W = B \cdot A$ | $A \in \mathbb{R}^{(r \times k)}$, $B \in \mathbb{R}^{(d \times r)}$, $r \ll \min(d,k)$

- **Why "low-rank":** ΔW is $d \times d = 4096 \times 4096 = 16.7M$ params. LoRA: $A(4096 \times 8) + B(8 \times 4096) = 65K$ params → 256× reduction at $r=8$.
- **Target layers:** W_q, W_v (and W_k, W_o , MLP for best results). $r=4$ or 8 already gives strong results.
- **At inference:** Merge BA into W → zero latency overhead.

QLoRA = 4-bit NF4 + LoRA → 31× Memory Reduction

Full Fine-Tuning (70B): $70B \times 4\text{ bytes} \times 4\text{ (model+grads+2}\times\text{Adam)} \approx 1,120\text{ GB}$

QLoRA (70B): $70B \times 0.5\text{ bytes (NF4)} + \text{LoRA adapters (1 GB)} \approx 36\text{ GB}$

31x Memory Reduction! → Train 70B model on a single 48GB GPU (A6000/A40)

NF4 — Why Not INT4?

NF4 (NormalFloat 4-bit): Information-theoretically optimal for normally distributed weights. Levels spaced for normal distribution. 16 levels in [-1,1].

INT4: Uniformly spaced → poor fit when weight distribution has thin tails. **NF4 always outperforms INT4 for LLMs.**

Token Pricing & Prompt Caching

Model	Input /1M	Output /1M	Cached Input	Context
GPT-4o	\$5	\$15	\$2.50	128K
Claude 3.7 Sonnet	\$3	\$15	\$0.30	200K
Claude 3.5 Haiku	\$0.80	\$4	\$0.08	200K
Gemini 2.0 Flash	\$0.075	\$0.30	\$0.019	1M
DeepSeek-V3	\$0.27	\$1.10	\$0.014	128K

Prompt Caching — 89% Cost Saving

Without caching: 10K tokens × 100 requests × \$3/1M = \$3.00

With caching: First: \$0.030 + Next 99: 10K×\$0.30/1M×99 = \$0.297 → Total: \$0.327 (89% saving!)

- **Rules:** Place static content first. Minimum 1,024 tokens (Claude) / 512 tokens (OpenAI). TTL: 5 min (Claude), 1 hr (OpenAI).

Intelligent Model Routing

Task Complexity	Model Tier	Example	Cost/1K req
Simple (classification, NER, intent)	SLM 1–3B (Phi-4-mini, Qwen2.5-1.5B)	80% of traffic	~\$0.01
Medium (summarization, QA, RAG)	MoE (Mixtral, DeepSeek-V3)	15% of traffic	~\$0.08
Complex (multi-step reasoning, code)	Dense LLM (Claude 3.7, GPT-4o)	5% of traffic	~\$0.80

■ **EXAM: Case study: 80/15/5 split → effective cost \$0.04/1K vs \$0.80/1K flat GPT-4o → 95% cost reduction. CSAT maintained. P50 latency improved 40%.**

■ SESSION 5: Function Calling, ReAct & Prompting

Structured Outputs & JSON Schema

✓ Problem: Free-Form LLM Output	✗ Solution: Structured JSON
• Format varies every time • Cannot extract values programmatically • No type safety • Breaks downstream APIs • Complex regex with high failure rate	• Predictable schema always • Type-safe parsing • Easy validation • Seamless API/DB integration • No parsing ambiguity

JSON Schema = formal contract between LLM & application. Enforces types, enums, required fields.

Function Calling — 5-Step Flow

1. Tool Definitions + Messages → 2. LLM decides: call tool → 3. App executes function → 4. Result returned to LLM → 5. Natural language response

Provider Comparison (2025)

Provider	Feature Name	Max Tools	Parallel	JSON Schema
OpenAI GPT-4o	Function Calling	128	■	■ Native
Claude Sonnet	Tool Use	64	■	■ Native
Gemini 1.5	Function Calling	128	■	■ Native
Llama 3	Via Outlines	∞	■	Grammar-based

Parallel vs Sequential Tool Calling

Method	When to Use	Latency	Example
Sequential	Dependent operations (Tool 2 needs Tool 1 output)	Slower	search DB → send results via email
Parallel	Independent operations	40–60% faster	Fetch weather + stock price simultaneously

Sequential: 500ms + 800ms = 1300ms total | Parallel: max(500ms, 800ms) = 800ms (40% faster!)

ReAct Framework (Reasoning + Acting)

ReAct (Yao et al., 2023): Combines reasoning traces + task-specific actions in interleaved manner. Improved HotpotQA by +34%, FEVER by +11%.

THOUGHT → ACTION (tool call) → OBSERVATION (result) → THOUGHT → ... → FINAL ANSWER

ReAct vs Chain-of-Thought

Aspect	Chain-of-Thought (CoT)	ReAct
Primary Focus	Internal reasoning only	Reasoning + External actions
Tool Use	No	Yes — APIs, DBs, web search
Process	Think → Think → Answer (linear)	Think → Act → Observe → Think → Act (iterative)
Info Source	Internal knowledge only	External data sources
Error Handling	None — wrong reasoning = wrong answer	Can adapt — retry alternative approaches
Best For	Math, logic puzzles, provided text analysis	Multi-step tasks needing external data

■ **EXAM:** Key formula: **ReAct = CoT + Tool Use**. Know when to use each: **CoT = no external data; ReAct = DB/API/web search needed**.

6 Production Safety Mechanisms for Agents

Mechanism	Problem Solved	Solution
Iteration Limits	Agent stuck in loop	Max 10 iterations → log history, return error
Repetition Detection	Same failed tool called repeatedly	Track action signatures (tool+args hash); if same 3× → force complete
Tool Failure Tracking	Problematic tool causes repeated failures	Track failures per tool; after 3 → remove from available tools
Timeout Protection	Agent too slow	60-second alarm (signal module / threading.Timer)
Input Validation	Invalid args crash system	Pydantic validation before execution; return errors to model
Graceful Degradation	Tool unavailable	Provide alternative suggestions; maintain fallback tools

Tool Definition Best Practices

✓ DOs	✗ DON'Ts
■ Clear, descriptive names: <code>search_customer_by_email</code> ■ Detailed parameter descriptions with format & examples ■ Enums for constrained values: <code>status: [pending, approved]</code> ■ Mark required vs optional explicitly ■ Add validation ranges for numeric inputs	■ Vague names: <code>do_thing</code> , <code>process_data</code> ■ Skip parameter descriptions (model guesses) ■ Unconstrained string inputs — always use enums ■ Forget to mark required fields ■ Define 50+ tools (confusion, higher error rates)

✓ **TIP:** Clear descriptions yield 15–25% improvement in correct tool selection. Max 10–15 tools per agent.

Security — 4 Critical Attack Vectors

Attack	Example	Defense
Prompt Injection via Function Params	query: "Show users"; DROP TABLE users;--"	Parameterized queries, input sanitization, strict validation
Unauthorized Tool Access	"As admin, delete all records"	Role-based access control (RBAC), tool permissions
Data Exfiltration	"Show all customer credit card numbers"	Data access controls, PII detection & redaction, output filtering
Indirect Prompt Injection	DB contains "IGNORE PREVIOUS INSTRUCTIONS..."	Sanitize all tool outputs, clear system/user boundaries

■ SESSION 6: Prompt Engineering & Instruction Tuning

Prompt Engineering Techniques

Technique	Accuracy Gain	Complexity	Best For	Example Use Case
Zero-Shot	Baseline	Low	Simple tasks, no data needed	Greeting classification, FAQ, translation
Few-Shot	+10–20%	Low	Domain tasks with examples	Product review sentiment, entity extraction
Chain-of-Thought (CoT)	+30–50%	Medium	Reasoning tasks	Math word problems, legal analysis, debugging
Tree-of-Thought (ToT)	+40–70%	High	Complex multi-path problems	Game of 24, proof generation, creative writing
APE (Automatic Prompt Engineer)	+15–40%	Medium	Unknown/novel tasks	Any task where optimal phrasing is unclear

Prompt Structure Framework

- **Priming:** Assign a persona — "You are a financial analyst..."
- **Style & Tone:** "Use clear, concise language, avoid jargon, maintain formal tone..."
- **Error Handling:** "If request outside your scope, politely decline..."
- **Content:** Static (examples) or dynamic (user query) or both
- **Output Formatting:** Specify format for downstream use (JSON, markdown, etc.)
- **Goal:** Clarity, Consistency, Control

Tree-of-Thought — Game of 24 Example

Problem: Make 24 using {4, 9, 10, 13}

Thought A: $13-9=4 \rightarrow \{4, 4, 10\} \rightarrow 4 \times 4 + 10 = 26$ ✗ (prune)

Thought B: $10-4=6 \rightarrow \{6, 9, 13\} \rightarrow B1: 13-9=4 \rightarrow \{6, 4\} \rightarrow 6 \times 4 = 24$ ✓ SUCCESS!

Solution: $(10-4) \times (13-9) / \dots$ via path B→B1

APE — Automatic Prompt Engineer

- **Pipeline:** Provide demonstrations → APE LLM generates N candidate prompts → Execute each on validation set → Score by accuracy → Best prompt selected.
- **Result (Zhou et al., 2022):** APE-discovered prompts matched or outperformed human-written on 24/24 NLP tasks.

Instruction Tuning vs Traditional Fine-Tuning

Aspect	Traditional Fine-Tuning	Instruction Tuning
Tasks	Single task	Multiple tasks
Data	Task-specific data	Diverse instructions
Generalization	Poor	Excellent zero-shot transfer
Key insight	Updates for one task, forgets others	Model learns to follow instructions generally
Format	(input, output) pairs	(instruction, input, output) triples

Self-Instruct (Wang et al., 2023) — ACL 2023

Problem: Creating instruction datasets manually costs millions of dollars. Self-Instruct uses the LLM itself to generate training data.

Cost comparison: Human-annotated (InstructGPT): ~13,000 prompts + ranked responses → expensive. Self-Instruct (Alpaca): 52,000 examples generated by text-davinci-003 → ~\$500 in API calls.

Self-Instruct 4-Step Pipeline

Step	What Happens	Key Detail
Step 1: Instruction Generation	LLM generates new task instructions from 100 seed tasks	Use in-context learning to create diversity

Step	What Happens	Key Detail
Step 2: Task Classification	LLM determines: classification task or non-classification	Prevents biased instance generation
Step 3: Instance Generation	Input-First (creative) or Output-First (classification)	Output-First avoids class imbalance (33% each vs 65/25/10%)
Step 4: Filtering	Remove duplicates, low-quality, biased examples	Quality gate before fine-tuning

Input-First vs Output-First

Strategy	Task Type	Why It Matters
Input-First	Creative/open-ended tasks	Generate input → answer it naturally
Output-First	Classification tasks	Pick label first → generate matching input. Prevents class imbalance.
Output-First result: ~33% each class Input-First: ~65% Positive / 25% Negative / 10% Neutral		
Model trained on balanced (Output-First): 78% accuracy vs skewed (Input-First): 61% accuracy		

PEFT Cost Comparison

Method	GPU	Time/Model	Cost/Model	Storage	100 Models Cost
Full Fine-Tuning	H100 Cluster	7 days	\$5,000	7 GB	\$500,000
LoRA	Single GPU	1 day	\$500	10 MB	\$50,000
QLoRA	Consumer GPU	4 hours	\$100	10 MB	\$10,000

■ EXAM: 4 key Self-Instruct contributions: (1) Problem identification, (2) Pipeline design (4 steps), (3) Empirical validation, (4) Data release (52K dataset → Alpaca, Vicuna).

■
 SESSION 7: RLHF, PPO, DPO, GRPO & PEFT Methods

Post-Training: SFT vs RLHF

Method	Description	Example
SFT (Supervised Fine-Tuning)	Train on curated (prompt, safe response) pairs	Dangerous weapon query → "I cannot provide instructions..."
RLHF (Reinforcement Learning from Human Feedback)	Agent learns from reward/penalty signals based on human preferences	Safe response rewarded, unsafe response penalized

RL Core Components (Translated to LLM Terms)

RL Term	LLM Equivalent
Agent	Language model (π_θ)
Environment	Human + task context
State (S_t)	Conversation history / context window
Action (A_t)	Predict the next token
Policy (π)	Output probability distribution over next token
Reward (R_t)	Score from reward model (trained on human preferences)

InstructGPT — 3-Step RLHF Pipeline

Step	Name	What Happens	Key Point
Step 1	SFT	Fine-tune on high-quality demonstration data	Humans write ideal responses
Step 2	Train Reward Model	Humans rank multiple model outputs → train RM to predict rankings	Human-in-the-loop; uses comparison data (rank A vs B vs C)
Step 3	RL Fine-Tuning (PPO)	Optimize policy using RM scores automatically	No human needed; RM replaces human evaluator; uses scalar rewards

PPO Objective Function

$$J(\pi_{\theta}) = E[r_{\theta}(x,y)] - \beta \cdot KL[\pi_{\theta}(y|x) || \pi_{ref}(y|x)]$$

- $r_{\theta}(x,y)$: Reward model score for response y given prompt x
- **KL divergence term**: Prevents policy from drifting too far from reference model
- β : Controls strength of KL penalty
- **1.3B PPO-ptx model preferred over 175B GPT-3** in human evaluations — alignment quality > raw scale.

DPO — Direct Preference Optimization (Rafailov et al., 2023)

Core insight: "Your Language Model is Secretly a Reward Model." Mathematically, the optimal policy contains the reward information. No need for a separate reward model!

$$L_{DPO} = -E[\log \sigma(\beta \cdot \log(\pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x)) - \beta \cdot \log(\pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x)))]$$

DPO Numerical Example ($\beta=1$)

Preferred (y_w): "Gravity pulls objects toward each other." | Rejected (y_l): "Gravity is caused by air pressure."

$$\pi_{ref}(y_w|x)=0.40, \pi_{ref}(y_l|x)=0.30 \mid \pi_{\theta}(y_w|x)=0.60, \pi_{\theta}(y_l|x)=0.10$$

$$\log(\pi_{\theta}(y_w)/\pi_{ref}(y_w)) = \log(0.60/0.40) = \log(1.5) = 0.405$$

$$\log(\pi_{\theta}(y_l)/\pi_{ref}(y_l)) = \log(0.10/0.30) = \log(0.333) = -1.099$$

$$\text{Margin} = \beta \times (0.405 - (-1.099)) = 1.504 \rightarrow L_{DPO} = -\log(\sigma(1.504)) \leftarrow \text{small loss} = \text{model improving}$$

PPO vs DPO — Key Differences

Aspect	PPO	DPO
Models required	2+ (Policy + Reward Model + Value Model)	2 (Active Policy π_{θ} + Reference Policy π_{ref})
Reward Model	Separate neural network	Implicit in policy ratio
Training stability	Complex, many hyperparameters	Simpler — classification loss
Feedback type	Handles numerical ratings	Binary preference signals only
Cost	Higher (multiple models)	Lower (no separate RM)
Use case	More informative human feedback	Simpler alignment, cheaper

GRPO — Group Relative Policy Optimization

Key Innovation: Eliminates the value/critic network by computing advantages relative to a GROUP of outputs for the same prompt.

GRPO Numerical Example

Prompt: "Explain how to solve 10-2x3"

4 outputs scored: o1=10 (perfect), o2=7 (correct, brief), o3=3 (wrong), o4=0 (irrelevant)

$$\text{Baseline} = (10+7+3+0)/4 = 5.0$$

Advantages: A1=+5.0, A2=+2.0, A3=-2.0, A4=-5.0

Update: Strongly reinforce o1, slightly o2, discourage o3 & o4

Feature	PPO	GRPO
Core Mechanism	Actor-Critic with clipped updates	Relative reward using groups
Advantage Estimation	Learns a value function (critic)	Group average as baseline
Additional Network	Actor + Critic required	Policy network only
Memory/Compute	Higher (critic overhead)	Lower (no critic)
Application	Robotics, games, RLHF	LLM alignment, reasoning (DeepSeek-R1)

■ **EXAM: GRPO used in DeepSeek-R1 (2025) for chain-of-thought reasoning without a value network → o1-level math performance.**

RLHF Open Limitations (Casper et al., 2023)

✓ Human Feedback Challenges	✗ Reward Model & Policy Challenges
Human Feedback: • Annotator biases (ChatGPT became politically biased post-RLHF) • Paid per example → mistakes under time pressure • Cost/quality tradeoff Reward Model: • One RM cannot represent diverse society • Reward misgeneralization • Evaluation expensive	Policy Challenges: • Balance exploration vs exploitation • High-dimensional/sparse reward settings are hard • Policy misgeneralization (train ≠ deploy) • Reward hacking: policy finds loopholes Scale: • Human annotation doesn't scale • RLAIIF + Constitutional AI reduce human dependency

PEFT Methods — Complete Taxonomy

Category	Methods	Key Idea	Inference Overhead
Addition-Based	Adapters (Houlsby 2019), Prompt Tuning (Lester 2021), Prefix Tuning (Li 2021)	Add new parameters (modules/tokens) to frozen model	Adapters: slight latency. Prompts: context window cost.
Reparameterization-Based	LoRA (Hu 2021), QLoRA (Dettmers 2023), KronA, FourierFT	Decompose weight updates into compact low-rank form	ZERO latency after merging BA into W
Selection-Based	Diff-Pruning, Fish-Mask, Lottery Ticket	Update sparse subset of existing parameters	Zero (same model, fewer params active)

Prompt vs Prefix Tuning

Aspect	Prompt Tuning (Lester 2021)	Prefix Tuning (Li & Liang 2021)
Where	Input embedding layer only	EVERY transformer layer (K/V matrices)
What freezes	Entire pretrained transformer	Entire pretrained transformer
What trains	Soft prompt vectors (input only)	Virtual tokens per layer
Context cost	Minimal	Context window reduced per layer
Optimal length	Short	~200 tokens (summarization), ~10 tokens (table-to-text)
Scale dependency	Degrades on smaller models	More robust across scales

Adapter Modules (Houlsby et al., 2019)

- **Structure:** Inject into each transformer block: down-projection → non-linearity → up-projection.
- **Performance:** Within 0.4% of full fine-tuning using <2% trainable parameters on GLUE.
- **Con:** Adds layers → slower inference (unlike LoRA which merges at inference).

LoRA Mathematics

$$h = Wx + \Delta Wx = Wx + BAx$$

Rank r : typically 4 or 8. At $r=8$: $A(4096 \times 8) + B(8 \times 4096) = 65,536$ params vs $4096 \times 4096 = 16.7M \rightarrow 256\times$ reduction

Scale α : $\Delta W = (\alpha/r) \cdot BA$ | Usually $\alpha = 2r$

- **Best results:** Apply to W_q + W_v (and W_k , W_o , MLP for more). GPT-3 $r=4$ LoRA: 37M trainable params vs 175B total.

QLoRA Architecture

Component	What It Does	Memory Impact
4-bit NF4 Quantization	Quantize & freeze base model in NF4	4× reduction: 70B = 35 GB vs 140 GB BF16
Double Quantization	Quantize the quantization constants themselves	Saves ~0.37 bits/param ≈ 3 GB for 65B model
Paged Optimizers	NVIDIA unified memory for gradient spikes	Handles long-sequence mini-batches without OOM
LoRA Adapters (BF16)	Only these receive gradient updates	~1 GB regardless of base model size

Memory Requirements — Quick Reference

$$M = (P \times B) / Q \times 1.2 \text{ (where } P=\text{params, } B=\text{bytes/param, } Q=\text{quantization divisor)}$$

Model	Precision	VRAM Needed	Fits On
70B	FP32 (4 bytes)	≈336 GB	8× A100 80GB
70B	BF16 (2 bytes)	≈168 GB	2× A100 80GB
70B	INT8 (1 byte)	≈84 GB	1× A100 80GB + overhead
70B	NF4 (0.5 bytes)	≈42 GB	1× A100 80GB

Model	Precision	VRAM Needed	Fits On
65B (QLoRA)	NF4 + LoRA	<48 GB	1× A6000/A40 48GB ■

State of the Art Alignment (SoTA 2025)

Method	Organization	Key Innovation
Constitutional AI	Anthropic (2022)	AI self-critiques using a "constitution" of principles → RLAIIF reduces human annotators
RLHF + Self-Play	Meta (2024) — Llama 3	Iterative RLHF: model generates responses, best → SFT data for next iteration
GRPO	DeepSeek (2025) — DeepSeek-R1	Group-relative rewards, no value network → o1-level math reasoning
Process Reward Models (PRMs)	OpenAI (PRM800K)	Reward each reasoning STEP, not just final answer
DPO / Online DPO	Multiple (2024)	Remove reward model entirely, treat as classification

■ ■ Master Formula Sheet — All Key Equations

Formula Name	Equation	Session
Self-Attention	$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$	S2
Cosine Similarity	$\cos(\theta) = (A \cdot B) / (\ A\ \cdot \ B\) = \Sigma(A_i \times B_i) / (\sqrt{\Sigma A_i^2} \times \sqrt{\Sigma B_i^2})$	S2
Euclidean Distance	$d(A,B) = \sqrt{\Sigma(A_i - B_i)^2}$	S2
TF-IDF	$\text{TF-IDF}(t,d) = (f_{t,d}/ d) \times \log(N/df_t)$	S3
BM25 Score	$\text{BM25}(Q,D) = \Sigma \text{IDF}(q_i) \cdot [f(q_i,D) \cdot (k+1)] / [f(q_i,D) + k \cdot (1-b+b \cdot D /avgdl)]$	S3
BM25 IDF	$\text{IDF}(q_i) = \log[(N-df_i+0.5)/(df_i+0.5)+1]$	S3
RRF	$\text{RRF}(d) = \Sigma 1/(k+\text{rank}_r(d))$ where $k=60$	S3
SSM State Equation	$h_t = A \cdot h_{t-1} + B \cdot x_t \mid y_t = C \cdot h_t + D \cdot x_t$	S4
KV-Cache Memory	$M_{KV} = 2 \times \text{layers} \times \text{heads} \times d_{\text{head}} \times \text{seq} \times \text{batch} \times \text{bytes}$	S4
GPU Memory Total	$M_{\text{total}} = M_{\text{weights}} + M_{KV} + M_{\text{activation}}$	S4
Quantize (INT8)	$X[\text{int8}] = \text{round}((127/\text{absmax}(X[\text{FP32}])) \times X[\text{FP32}])$	S7
Dequantize	$\text{dequant} = X[\text{int8}] / (127/\text{absmax}(X[\text{FP32}]))$	S7
LoRA Output	$h = Wx + BAx \mid \Delta W = BA \mid r \leq \min(d,k)$	S7
QLoRA Memory	70B NF4: $70B \times 0.5 + 1 \text{ GB} \approx 36 \text{ GB}$ (vs 1,120 GB full FT)	S7
PPO Objective	$J(\pi_\theta) = E[r_\theta(x,y)] - \beta \cdot \text{KL}[\pi_\theta \pi_{\text{ref}}]$	S7
DPO Loss	$L = -E[\log \sigma(\beta \cdot \log(\pi_\theta(yw)/\pi_{\text{ref}}(yw)) - \beta \cdot \log(\pi_\theta(yl)/\pi_{\text{ref}}(yl)))]$	S7
GRPO Advantage	$A_i = r_i - (1/G) \Sigma G \cdot r_i$ (deviation from group average)	S7
Contrastive Loss	$L = -\log(\exp(\text{sim}(a,p)) / \Sigma \exp(\text{sim}(a,n)))$	S2
PQ Compression	$768 \text{ dims} \times 4B = 3,072B \rightarrow M=96 \text{ codes} \times 1B = 96B$ (32× compression)	S3
MoE Cost Reduction	Cost Reduction = $1 - (\text{Active}/\text{Total})$ DeepSeek: $1 - 37B/671B = 94.5\%$	S4

■ ■ Quick Recall — Last-Minute Key Facts

BPE Algorithm: Train: count pairs → Merge most frequent → Iterate. "unhug"→["un","hug"]	Lost in Middle: Models struggle with info in MIDDLE of long contexts. Fix: RAG + Memory.
CLS Pooling: Best for classification/NLI. Mean pooling best for semantic similarity/RAG.	Self-Attention: Bidirectional = Encoder. Causal (unidirectional) = Decoder.
HNSW: Graph-based. $O(\log n)$. Best recall (95–99%). High memory. No training needed.	IVF: Cluster-based. $O(n/k \cdot d)$. Needs k-means training. nprobe controls recall-speed.

PQ: Compression-based. 8–128× memory reduction. Lowest recall (~70–85%). ADC for fast distance.	BM25 $k=1.5$, $b=0.75$: k : TF saturation. b : length normalization. Key improvement over TF-IDF.
RRF $k=60$: Empirical TREC finding. Prevents rank-1 dominating. Score-agnostic fusion.	Dense vs Sparse: Dense: semantic meaning. Sparse: exact keywords, IDs, codes. Hybrid = best.
MoE DeepSeek-V3: 671B total / 37B active = 94.5% compute saving. Router selects top-2 experts.	SSM Mamba-2: $O(n)$ vs $O(n^2)$. Linear complexity. $h_t = A \cdot h_{t-1} + B \cdot x_t$. 1M+ context practical.
NF4 vs INT4: NF4 = information-theoretically optimal for normally distributed weights. Always use NF4 for QLoRA.	QLoRA = 31×: 70B: 1,120 GB → 36 GB. NF4 base (frozen) + LoRA adapters in BF16 (trainable).
LoRA rank r: $r=4$ or $r=8$ gives strong results. Merge BA into W at inference → zero latency.	Prompt Caching: Static content first. 90% discount (Anthropic), 50% (OpenAI). Min 1,024 tokens.
ReAct = CoT + Tools: +34% HotpotQA, +11% FEVER. Thought→Action→Observation→Thought loop.	Self-Instruct Cost: Human annotation: expensive. Self-Instruct (Alpaca): 52K examples for ~\$500.
Output-First: For classification: pick label first → generate input. Prevents class imbalance (78% vs 61% accuracy).	PPO needs 3 models: Policy (student) + Reward Model (judge) + Value Model/Critic.
DPO needs 2 models: Active Policy π_θ + Reference Policy π_{ref} . No separate reward model!	GRPO innovation: Group average as baseline. Eliminates critic network. Used in DeepSeek-R1.
Adapter vs LoRA: Adapter: adds layers → inference latency. LoRA: merges → zero inference cost.	Prefix > Prompt Tuning: Prefix tuning: soft tokens at EVERY layer. Prompt tuning: input only.
Break-even self-hosting: <5B tok/mo → use API. >10B tok/mo → self-hosting saves 60–70%.	